

PROJECT REPORT

Student Performance Tracker

A Python Flask & SQLite Academic Web Application

Course:	Bachelor of Computer Applications (BCA)
Subject:	Software Engineering / Mini Project
Technology Stack:	Python 3, Flask Web Framework, SQLite3 Database, HTML5/CSS3
Author / Developer:	BCA Student
Deployment Platform:	Render Cloud Platform
Date:	July 2026

Submitted for Academic Evaluation

1. Executive Summary & Objectives

The **Student Performance Tracker** is a lightweight, robust Python Flask web application designed to help educational institutions and teachers digitally record, analyze, and manage student academic performance. By automating grade accumulation and average percentage calculations, the system eliminates manual record-keeping errors and simplifies student progress monitoring.

Key Objectives of the System:

- **Digital Information Management:** Store student names and unique roll numbers in a centralized database.
- **Subject-Wise Grade Tracking:** Record individual subject marks with strict range validation (0 to 100).
- **Automated Average Computation:** Instantly compute individual and class-wide academic averages.
- **Clean User Interface:** Provide an intuitive, responsive web portal accessible across modern browsers.
- **Modular Software Architecture:** Demonstrate core Object-Oriented Programming (OOP) concepts coupled with relational SQLite storage.

2. System Architecture & Technologies

Component	Technology / Tool	Purpose in System
Programming Language	Python 3.x	Core application logic and object orientation
Web Framework	Flask 3.0	HTTP Routing, Request Handling, Jinja2 Rendering
Database Engine	SQLite 3	Lightweight, zero-config relational database storage
Frontend Technologies	HTML5 & CSS3	Responsive web interface and student entry forms
Deployment Server	Gunicorn & Render	Cloud platform hosting and production execution

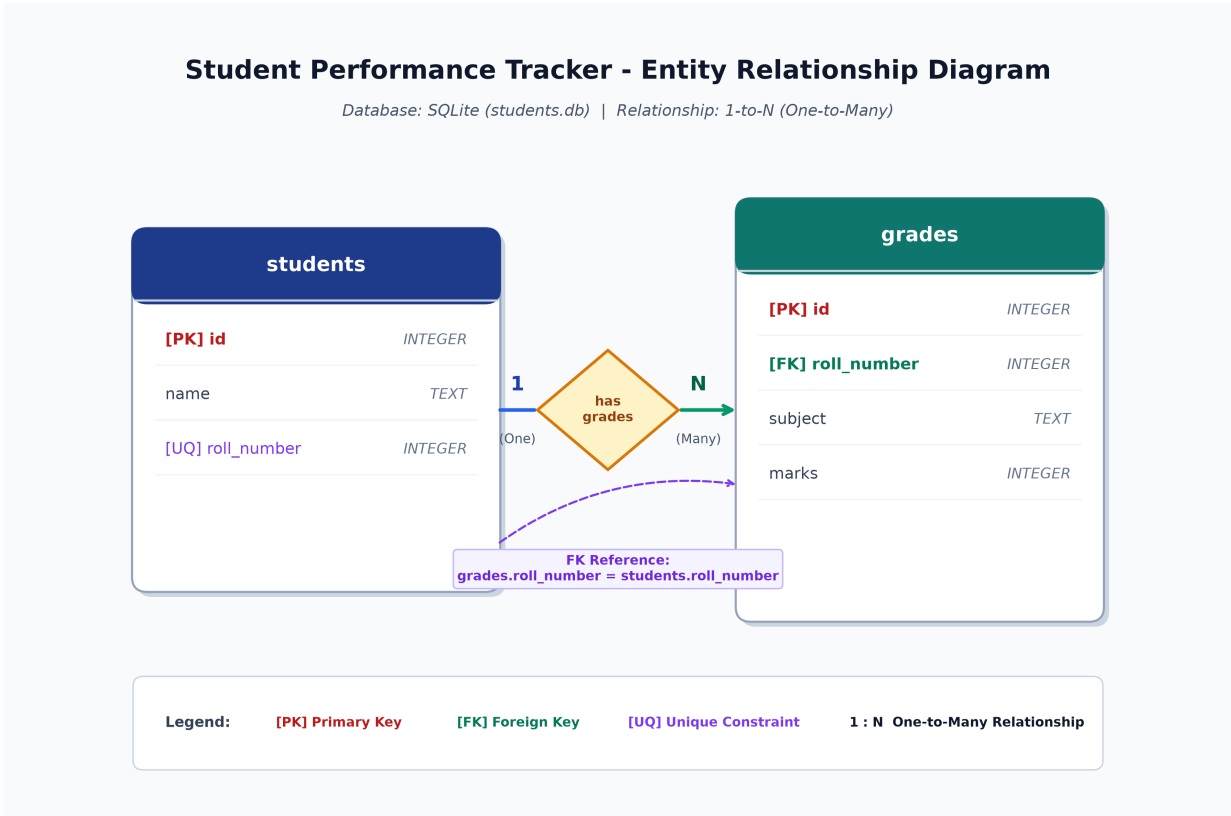
3. Database Design & Relational Schema

The application relies on SQLite (students.db) for persistent storage. The schema consists of two primary normalized tables linked by a 1-to-N (One-to-Many) foreign key relationship:

1. **students Table:** Stores master student records. roll_number is enforced with a UNIQUE constraint.

2. **grades Table:** Stores subject-wise marks. References students.roll_number to link grades to their respective student.

Figure 3.1: Entity-Relationship (ER) Diagram



4. Object-Oriented Software Architecture

The application architecture strictly adheres to modular separation of concerns. Below is the detailed breakdown of the system modules:

4.1 Core Domain Model (student.py)

Defines the Student class representing an individual student entity. encapsulates name, roll number, and a dictionary of subject-mark pairs.

```
class Student:
    def __init__(self, name, roll_number):
        self.name = name
        self.roll_number = roll_number
        self.grades = {} # e.g., {'Math': 90, 'Science': 85}

    def add_grade(self, subject, marks):
        self.grades[subject] = marks

    def calculate_average(self):
        if not self.grades: return 0
        return sum(self.grades.values()) / len(self.grades)
```

4.2 Business Manager (tracker.py)

Defines the StudentTracker class which acts as an in-memory manager containing a collection of Student objects, supporting search by roll number and aggregate reporting.

4.3 Data Access Layer (database.py)

Encapsulates raw SQLite SQL operations. Provides reusable helper functions including add_student_db(), add_grade_db(), get_student_db(), and get_all_students_db().

4.4 Web Controller Layer (app.py)

Flask routes managing HTTP GET and POST requests. Validates mark boundaries (0-100), handles duplicate roll number exceptions gracefully, and renders HTML templates.

5. System Testing & Quality Assurance

Test ID	Feature Under Test	Input Data	Expected Result	Status
TC-01	Add Valid Student	Name: Rahul, Roll: 101	Student added successfully	PASS
TC-02	Duplicate Roll Check	Roll: 101 (existing)	Display error 'Roll Number Already Exists'	PASS

TC-03	Grade Range Validation	Marks: 150 (out of range)	Reject input: 'Marks should be 0-100'	PASS
TC-04	Individual Average	Math: 90, Science: 80	Calculated Average: 85.0	PASS
TC-05	Class Average Report	Multiple students in DB	Render complete table with roll & average	PASS

6. Conclusion & Future Enhancements

The **Student Performance Tracker** successfully demonstrates the application of Python, Flask web microframework, SQLite database design, and Object-Oriented principles to solve a real-world educational problem. The application is production-ready for lightweight deployments.

Recommended Future Scope:

1. **Role-Based Authentication:** Secure logins for Teachers (write access) and Students (read-only portal).
2. **Automated PDF Report Cards:** Generate downloadable PDF marksheets for individual students.
3. **Subject Topper Analytics:** Identify highest scorers per subject automatically.
4. **Cloud Database Upgrade:** Migrate SQLite to PostgreSQL / MySQL for high concurrency.